Análise Comparativa de Estratégias de Controle Populacional em Algoritmos Genéticos Aplicados ao Problema das N-Rainhas

Vinícius Rubin[1], **Igor Yepes**[1]

[1]Instituto Federal de Educação, Ciência e Tecnologia Farroupilha – Campus Frederico Westphalen (IFFAR-FW)

Abstract. *This article investigates how different population control strategies affect the performance of genetic algorithms applied to the n-queens problem. The study presents the design of a baseline implementation, details the experimental protocol, and discusses the interpretation of metrics such as execution time, number of generations, and success rate. The objective is to identify configurations capable of preserving population diversity while accelerating convergence, offering practical guidelines for applying evolutionary heuristics to combinatorial search problems.*
Keywords: *Genetic Algorithms, Population Control, Evolutionary Computation, N-Queens Problem.*

Resumo. *Este artigo investiga como diferentes estratégias de controle populacional influenciam o desempenho de algoritmos genéticos aplicados ao problema das n-rainhas. São apresentados o planejamento da implementação base, o protocolo experimental e a análise de métricas como tempo de execução, número de gerações e taxa de sucesso. O objetivo é identificar configurações capazes de preservar a diversidade populacional e, ao mesmo tempo, acelerar a convergência, fornecendo orientações práticas para o uso de heurísticas evolutivas em tarefas de busca combinatória.*
Palavras-chave: *Algoritmos Genéticos, Controle Populacional, Computação Evolutiva, Problema das N-Rainhas.*

1. Introdução

Algoritmos Genéticos (AGs) são métodos de busca inspirados nos mecanismos de evolução natural, amplamente empregados na resolução de problemas complexos devido à sua capacidade de equilibrar exploração estocástica e intensificação por meio de operadores simples de reprodução [Eiben and Smith 2015, Holland 1975]. Esses algoritmos têm sido aplicados em diversas áreas — de problemas combinatórios clássicos a sistemas de engenharia — e sua eficiência depende de um conjunto de decisões de projeto que afetam diretamente o processo evolutivo.

Entre esses componentes, o controle populacional destaca-se como um dos fatores menos padronizados e, ao mesmo tempo, mais influentes no comportamento dos AGs. Esse mecanismo determina como indivíduos são preservados, substituídos ou eliminados ao longo das gerações, afetando diversidade, pressão seletiva, estabilidade evolutiva e risco de convergência prematura. Embora estratégias como substituição total, substituição parcial, elitismo e técnicas baseadas em ciclos de vida sejam amplamente relatadas na

literatura, comparações diretas entre elas são raras. Os estudos existentes geralmente variam simultaneamente representação, operadores ou configurações experimentais, o que dificulta isolar o impacto específico da política de substituição.

Essa ausência de padronização configura uma lacuna importante na área: ainda não está claro, sob condições controladas, como diferentes estratégias de controle populacional influenciam taxa de sucesso, custo computacional e profundidade de busca em AGs. Assim, surge a seguinte pergunta de pesquisa:

> *Como diferentes estratégias de controle populacional afetam o desempenho de Algoritmos Genéticos quando todos os demais componentes do algoritmo permanecem constantes?*

Para investigar essa questão, este trabalho realiza uma comparação sistemática entre quatro mecanismos de controle populacional — substituição total, substituição parcial, elitismo e uma abordagem baseada em idade inspirada no modelo GAVaPS — mantendo fixos representação, função de aptidão, seleção, cruzamento e mutação. O problema das n-rainhas é adotado como cenário experimental por constituir um benchmark clássico, escalável e amplamente utilizado na avaliação de heurísticas evolutivas.

A contribuição principal deste estudo consiste em fornecer uma análise empírica rigorosa e reprodutível sobre o efeito de diferentes políticas de substituição na dinâmica evolutiva dos AGs. Especificamente, o trabalho:

1. compara, sob condições uniformes, o desempenho de quatro estratégias de controle populacional;
2. avalia como cada mecanismo influencia taxa de sucesso, número de gerações, tempo de execução e limites de parada;
3. discute vantagens, limitações e cenários de aplicação de cada abordagem.

Os experimentos incluem múltiplas execuções independentes por configuração e utilizam critérios híbridos de parada (tempo, geração máxima e solução encontrada), possibilitando medir médias, variâncias e padrões de comportamento evolutivo. Essa estrutura fornece evidências quantitativas que auxiliam na escolha da estratégia de controle populacional mais adequada para problemas combinatórios de natureza semelhante.

2. Fundamentação Teórica

A fundamentação teórica deste trabalho apresenta os conceitos necessários para compreender o funcionamento de Algoritmos Genéticos (AGs), sua aplicação ao problema das n-rainhas e o papel das estratégias de controle populacional na dinâmica evolutiva. Inicialmente, são discutidos os princípios básicos dos AGs, com ênfase em seus componentes estruturais. Em seguida, descreve-se o fluxo geral de um AG clássico, por meio de um pseudocódigo que servirá de referência conceitual para todas as variantes analisadas. Depois, revisita-se o problema das n-rainhas como cenário experimental amplamente utilizado na literatura. São também apresentados os principais elementos do ambiente computacional adotado, com foco em bibliotecas numéricas para experimentação reprodutível. Por fim, são detalhadas as estratégias de controle populacional e os trabalhos relacionados que motivam o recorte específico deste estudo.

2.1. Algoritmos Genéticos

Algoritmos Genéticos (AGs) são meta-heurísticas inspiradas em princípios de evolução biológica, nas quais uma população de soluções candidatas é iterativamente modificada por operadores de seleção, cruzamento e mutação [Eiben and Smith 2015, Holland 1975]. Cada indivíduo é avaliado por uma função de aptidão (*fitness*), e o processo evolutivo tende a favorecer configurações de melhor desempenho ao longo das gerações, compondo um ciclo de variação e seleção que equilibra exploração e intensificação no espaço de busca.

O comportamento de um AG é fortemente influenciado por três grupos de decisões de projeto [de Lacerda and de Leon Ferreira de Carvalho 1999]: (i) a forma de representação dos indivíduos (ou codificação das soluções), (ii) a função de avaliação (função de aptidão) e (iii) os operadores evolutivos e a política de substituição populacional.

A literatura tradicional em AGs dedica grande atenção à escolha da representação (binária, inteira, permutacional, real, entre outras) e ao desenho de operadores de cruzamento e mutação adequados ao problema [Goldberg 1989, Eiben and Smith 2015]. Em contraste, o *controle populacional* — isto é, a forma como indivíduos são mantidos, substituídos, removidos ou preservados ao longo das gerações — costuma ser tratado como um detalhe de implementação, apesar de influenciar diretamente aspectos como diversidade, pressão seletiva, convergência e risco de aprisionamento em ótimos locais [Whitley 1994, Mitchell 1996].

Neste trabalho, representação, função de aptidão, seleção, cruzamento e mutação são mantidos fixos, e o foco recai exclusivamente sobre o componente de controle populacional. Essa escolha permite isolar e mensurar o impacto de diferentes políticas de substituição no desempenho global do algoritmo, mantendo constantes os demais fatores que tradicionalmente afetam a qualidade da busca.

2.2. Fluxo Geral de um Algoritmo Genético

Embora existam inúmeras variantes, o fluxo de execução de um AG tradicional segue uma estrutura geral bem estabelecida, composta por inicialização da população, avaliação, reprodução e atualização da população ao longo de múltiplas gerações [Eiben and Smith 2015, Mitchell 1996]. Critérios de parada usuais incluem: (i) número máximo de gerações, (ii) orçamento máximo de avaliações da função de aptidão e (iii) limite de tempo de execução.

O pseudocódigo apresentado a seguir resume o processo e serve como modelo conceitual sobre o qual as diferentes estratégias de controle populacional serão discutidas. No contexto deste trabalho, o critério de parada é posteriormente especializado para um limite máximo de avaliações (ver Seção 3).

Algorithm 1 Pseudocódigo de um Algoritmo Genético Clássico

Entrada: tamanho populacional N, orçamento máximo de avaliações $A_{\max}$, operadores genéticos
Saída: melhor solução encontrada

Inicializar população P com N indivíduos
Avaliar P e inicializar contador de avaliações $A \leftarrow N$
while não satisfeito o critério de parada **e** $A < A_{\max}$ **do**
// Seleção e Reprodução
    Selecionar pais usando torneio (ou outro método)
    Aplicar cruzamento para gerar descendentes
    Aplicar mutação aos descendentes
    Avaliar descendentes e atualizar A
// Controle Populacional
    Combinar populações conforme a estratégia escolhida:
    – substituição total, ou
    – substituição parcial, ou
    – elitismo, ou
    – controle por idade (GAVaPS/Michalewicz)
    Atualizar população P
end while
Retornar melhor indivíduo de P

Esse pseudocódigo é utilizado como referência conceitual para as variações testadas neste estudo, alterando-se apenas a estratégia de controle populacional aplicada em cada experimento, enquanto os demais componentes permanecem inalterados. A especialização para um critério de parada baseado exclusivamente em avaliações é detalhada na Seção 3.

2.3. O Problema das *n*-Rainhas

O problema das *n*-rainhas consiste em posicionar *n* rainhas em um tabuleiro de xadrez de dimensões *n*×*n* de forma que nenhuma delas ataque outra, o que implica ausência de conflitos em linhas, colunas e diagonais [Bell and Stevens 2009]. Embora a formulação seja simples, o número de configurações possíveis cresce rapidamente com *n*, tornando abordagens exaustivas inviáveis para instâncias maiores e caracterizando um problema combinatório de difícil exploração.

Estudos como o de Ali et al. [Ali et al. 2023] mostram que buscas clássicas (BFS/DFS) são eficientes apenas para instâncias pequenas, ao passo que, para valores maiores de *n*, meta-heurísticas, como AGs, tornam-se alternativas mais competitivas. Essa combinação de estrutura simples, dificuldade ajustável e ampla presença na literatura torna o problema das *n*-rainhas um *benchmark* natural para avaliar variações de AGs [Sarkar and Nag 2017, Farhan et al. 2015].

Neste trabalho, o problema das *n*-rainhas é adotado como cenário experimental para comparar estratégias de controle populacional, beneficiando-se de uma representação permutacional que elimina conflitos de linha e coluna e concentra a função de aptidão na

contagem de conflitos diagonais. Essa característica contribui para reduzir o impacto de soluções inviáveis e realçar o efeito das diferentes políticas de substituição populacional.

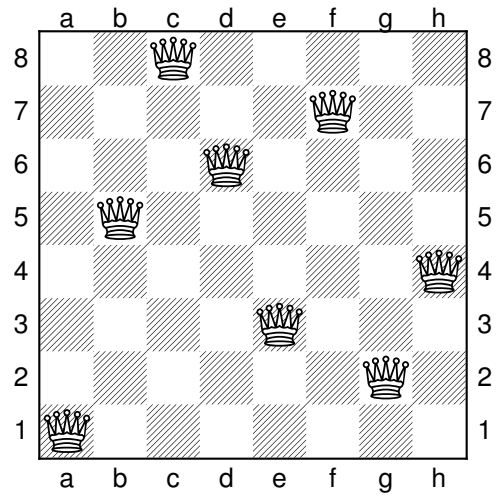


Figura 1. Exemplo de solução válida para o problema das 8-rainhas.

2.4. Uso da Linguagem Python e Bibliotecas Relacionadas

A implementação experimental foi desenvolvida em `Python`, linguagem amplamente adotada em computação científica pela clareza sintática e pelo ecossistema consolidado de bibliotecas numéricas [Python Software Foundation 2025a]. Essa escolha facilita a construção de protótipos, a automatização de experimentos e a reprodutibilidade dos resultados.

A biblioteca `NumPy` foi utilizada para operações vetorizadas e para a representação eficiente da população [Harris et al. 2020]. A geração de números aleatórios baseou-se em `numpy.random` [Python Software Foundation 2025b], garantindo controle explícito de sementes e replicação de cenários. A visualização dos resultados foi realizada com `matplotlib` [Hunter 2007], permitindo a construção de gráficos para análise comparativa das estratégias de controle populacional.

Esse conjunto de ferramentas permite concentrar o esforço no desenho dos experimentos e na análise do comportamento dos AGs, reduzindo interferências de infraestrutura e favorecendo a reprodutibilidade.

2.5. Técnicas de Controle Populacional Abordadas Neste Trabalho

Este trabalho considera quatro estratégias de controle populacional aplicadas sobre o mesmo AG base: substituição total, substituição parcial, elitismo e controle por idade. Em todas elas, representação, função de aptidão, método de seleção, cruzamento e mutação são mantidos fixos. Dessa forma, torna-se possível isolar o efeito da política de substituição sobre a dinâmica evolutiva, em particular sobre diversidade, convergência e risco de aprisionamento em ótimos locais [Holland 1975, Goldberg 1989, Eiben and Smith 2015, Whitley 1994]. As estratégias avaliadas diferem principalmente em dois aspectos: (i) a fração da população renovada em cada geração

e (ii) o critério utilizado para decidir quais indivíduos permanecem ou são removidos [de Lacerda and de Leon Ferreira de Carvalho 1999, Mitchell 1996].

2.5.1. Substituição Total

Na substituição total, a cada geração todos os indivíduos da população corrente são substituídos pelos descendentes recém-gerados. Em termos operacionais, o algoritmo:

1. seleciona pares de pais na população atual;
2. aplica cruzamento e mutação para produzir uma nova população de tamanho N;
3. descarta integralmente a população anterior.

Trata-se do esquema mais simples e amplamente utilizado em AGs, pois facilita a implementação e o controle de parâmetros [Goldberg 1989, Whitley 1994]. No entanto, por descartar toda a geração anterior, tende a reduzir a diversidade rapidamente, tornando o algoritmo mais suscetível à convergência prematura, especialmente quando a pressão seletiva é alta ou a mutação é conservadora [De Jong 1975, Spears 1993]. A substituição total é frequentemente utilizada como referência de comparação para outras políticas de controle populacional.

2.5.2. Substituição Parcial

Na substituição parcial, apenas uma fração da população é renovada a cada geração, caracterizando um modelo do tipo *steady-state* [Eiben and Smith 2015, Whitley 1994]. De forma geral, o processo segue os passos:

1. a partir da população atual, selecionam-se pais e produzem-se alguns descendentes;
2. esses descendentes são inseridos na população;
3. um subconjunto de indivíduos é removido, preservando o tamanho populacional.

A fração de indivíduos substituída em cada geração controla o ritmo de renovação: valores menores favorecem uma evolução mais gradual, enquanto valores maiores aproximam o comportamento do modelo geracional [Goldberg 1989, Mitchell 1996].

Na literatura, diferentes critérios podem ser utilizados para decidir quais indivíduos permanecem ou são removidos (por aptidão, aleatoriedade, idade, entre outros). Neste trabalho, a estratégia de substituição parcial preserva uma fração fixa de indivíduos escolhidos **aleatoriamente**, sem preferência explícita pelos mais aptos. O elitismo, por sua vez, é tratado como uma estratégia distinta, responsável por garantir a preservação sistemática dos melhores indivíduos.

2.5.3. Elitismo

O elitismo introduz um mecanismo explícito de preservação dos melhores indivíduos [Goldberg 1989, Whitley 1994]. Em cada geração:

1. uma pequena fração dos indivíduos com melhor aptidão é copiada diretamente para a próxima geração;

2. o restante da população é preenchido com descendentes produzidos pelos operadores de cruzamento e mutação;
3. indivíduos menos aptos são descartados para manter o tamanho populacional.

Esse mecanismo garante que as melhores soluções encontradas não sejam perdidas por efeito do acaso, o que tende a acelerar a convergência [De Jong 1975, Goldberg 1989]. Por outro lado, a preservação sistemática de indivíduos muito aptos pode reduzir a diversidade e acentuar o risco de estagnação em ótimos locais, sobretudo quando a população é pequena [Eiben and Smith 2015, de Lacerda and de Leon Ferreira de Carvalho 1999]. Assim, o elitismo representa um reforço da pressão seletiva que pode ser benéfico ou prejudicial, dependendo do equilíbrio com outros componentes do algoritmo.

2.5.4. Controle Populacional por Idade (GAVaPS / Michalewicz)

A estratégia de controle por idade é inspirada no modelo *Genetic Algorithms with Varying Population Size* (GAVaPS) [Michalewicz 1994]. Nesse esquema, cada indivíduo passa a carregar, além do cromossomo e da aptidão, duas informações adicionais:

- sua idade corrente (número de gerações que já sobreviveu);
- um tempo de vida máximo (*lifetime*), definido no momento de seu nascimento.

A ideia central é que indivíduos mais aptos recebam um *lifetime* maior, podendo permanecer mais tempo na população, enquanto indivíduos menos aptos tendem a ser descartados mais rapidamente [Michalewicz 1994]. Em GAVaPS, esse *lifetime* é tipicamente calculado a partir da aptidão relativa do indivíduo em relação às estatísticas da população (mínimo, máximo e média), por meio de uma função (frequentemente bi-linear) que mapeia a qualidade de $f(i)$ para o intervalo discreto $[L_{\min}, L_{\max}]$. Em termos qualitativos, indivíduos com aptidão melhor (mais próximos do melhor valor observado) recebem tempos de vida próximos de $L_{\max}$, enquanto indivíduos piores aproximam-se de $L_{\min}$.

Em cada geração, o processo segue, de forma simplificada, as etapas [Michalewicz 1994, Eiben and Smith 2015]:

1. incrementar a idade de todos os indivíduos;
2. remover da população aqueles cuja idade ultrapassou seu *lifetime*;
3. a partir dos sobreviventes, gerar novos descendentes (via seleção, cruzamento e mutação) em quantidade proporcional ao tamanho atual da população, permitindo que o número total de indivíduos possa crescer ou encolher dinamicamente.

Diferentemente das políticas clássicas, em que a remoção é determinada principalmente pela aptidão instantânea, aqui a exclusão ocorre por envelhecimento: mesmo indivíduos muito aptos eventualmente deixam a população ao atingir seu limite de vida. Esse mecanismo introduz uma *diversidade temporal*, evitando que poucas soluções dominem a população indefinidamente e favorecendo um equilíbrio entre intensificação (preservar bons indivíduos por mais tempo) e exploração (renovar continuamente a população) [Michalewicz 1994].

No contexto deste trabalho, o controle por idade é comparado diretamente com esquemas de substituição total, substituição parcial e elitismo, buscando avaliar em que medida a introdução do *lifetime* e da variação de tamanho populacional altera a taxa

de sucesso, o esforço computacional (medido em número de avaliações) e o comportamento evolutivo de AGs aplicados ao problema das *n*-rainhas [Richter and Berlezi 2012, Sarkar and Nag 2017, Ali et al. 2023].

2.6. Trabalhos Relacionados

Diversos trabalhos utilizam o problema das *n*-rainhas como *benchmark* para avaliar variações em AGs, explorando principalmente representação e mecanismos de controle da diversidade. Nesta subseção, são destacados estudos que motivam o desenho experimental adotado e ajudam a situar a contribuição deste trabalho, que se concentra no impacto de diferentes estratégias de controle populacional.

2.6.1. Representações Binária e Inteira para o Problema das *n*-Rainhas [Richter and Berlezi 2012]

Richter et al. [Richter and Berlezi 2012] comparam duas representações genéticas para o problema das *n*-rainhas: uma codificação binária e uma codificação inteira do tipo *InOrder*. A primeira exige conversão de bits para posições de rainhas e é suscetível a colisões em linhas; a segunda mapeia diretamente cada coluna a uma linha distinta, evitando conflitos de linha e coluna por construção.

Os resultados mostram que a representação *InOrder* reduz o número de soluções inviáveis e acelera a convergência, sobretudo para tabuleiros maiores. Embora os autores mantenham o controle populacional em um esquema tradicional, seu trabalho reforça a importância de fixar uma representação adequada antes de comparar outras variações de um AG. Este estudo adota uma representação por permutação semelhante à *InOrder* justamente para minimizar o impacto de soluções inviáveis e isolar o efeito das estratégias de substituição populacional.

2.6.2. Comparação entre Algoritmo Genético e Buscas Clássicas (BFS/DFS) [Ali et al. 2023]

Ali et al. [Ali et al. 2023] comparam um AG com buscas clássicas (BFS/DFS) no problema das *n*-rainhas, analisando a escalabilidade de cada método. Utilizando uma representação vetorial em que cada posição do cromossomo indica a linha ocupada por uma rainha, os autores observam que, para instâncias pequenas, BFS e DFS são mais eficientes em tempo de execução, mas o AG passa a ser mais competitivo à medida que o tamanho do tabuleiro aumenta.

Embora o foco do estudo seja a comparação entre paradigmas (busca exaustiva vs. AG), os resultados reforçam o papel dos AGs como alternativa viável para instâncias maiores e justificam seu uso como meta-heurística de referência. Diferentemente de Ali et al., o presente trabalho não compara AGs com métodos clássicos de busca, mas explora, dentro do próprio paradigma evolutivo, como diferentes políticas de controle populacional afetam o desempenho em um problema combinatório já consolidado na literatura.

2.6.3. Algoritmo Genético Adaptativo com Controle de Diversidade [Sarkar and Nag 2017]

Sarkar e Nag [Sarkar and Nag 2017] propõem um AG adaptativo para o problema das *n*-rainhas, enfatizando mecanismos explícitos de controle da diversidade populacional. Os autores utilizam uma representação por permutação, um cruzamento do tipo *Order 1 Crossover* e operadores de mutação com probabilidades relativamente altas, além de estruturas auxiliares (como um repositório de indivíduos menos aptos) para evitar estagnação em ótimos locais.

Os resultados indicam que o esquema adaptativo encontra soluções ótimas para valores elevados de N (até 100), sugerindo que a forma como a população é renovada tem influência direta na qualidade e na velocidade da convergência. O trabalho, porém, combina várias alterações simultâneas (função de aptidão, mutação, repositório, etc.), o que dificulta isolar o efeito de cada componente. Em contraste, o presente estudo mantém constantes a representação e os operadores genéticos e varia apenas a política de controle populacional, buscando medir de forma mais direta o impacto da substituição e do envelhecimento sobre o desempenho do AG.

2.6.4. Busca de Todas as Soluções Válidas para as 8-Rainhas [Farhan et al. 2015]

Farhan et al. [Farhan et al. 2015] utilizam um AG para encontrar as 92 soluções válidas do problema das 8-rainhas, empregando uma representação vetorial que elimina conflitos em linhas e colunas e concentra a avaliação em ataques diagonais. A função de aptidão é definida em termos do número de conflitos diagonais, e o AG é executado até que todas as soluções distintas sejam encontradas.

Embora o objetivo seja diferente do deste trabalho (cobertura do espaço de soluções em vez de análise comparativa de políticas de controle), o estudo ilustra a capacidade dos AGs de explorar um espaço combinatório rico e de encontrar múltiplas soluções válidas. Aqui, essa característica é aproveitada para investigar como diferentes estratégias de substituição e um esquema de controle por idade alteram a taxa de sucesso e o esforço necessário para encontrar ao menos uma solução válida em instâncias progressivamente mais difíceis.

3. Metodologia

Esta seção descreve o desenho experimental empregado para comparar quatro estratégias de controle populacional em Algoritmos Genéticos (AGs) aplicados ao problema das *n*-rainhas. O estudo segue diretrizes de experimentação reprodutível em computação evolutiva [Eiben and Smith 2015], mantendo constantes todos os componentes do algoritmo que não estão diretamente relacionados ao mecanismo de substituição populacional. Dessa forma, o impacto de cada estratégia pode ser analisado isoladamente, sem interferências provenientes de diferenças estruturais no operador evolutivo.

3.1. Algoritmo Genético Base

A representação utilizada é uma permutação de tamanho n, onde a posição i do vetor indica a linha ocupada pela rainha na coluna i, garantindo a ausência de conflitos em

linhas e colunas. Assim, a função de aptidão consiste exclusivamente no número de pares de rainhas em conflito diagonal, métrica de uso amplamente consolidado na literatura e de complexidade $O(n^2)$.

A população inicial é gerada embaralhando sequências de 0 a $n-1$. O algoritmo utiliza:

- **Seleção**: torneio binário;
- **Cruzamento**: *Order Crossover* (OX), aplicado em lote;
- **Mutação**: troca de duas posições (*swap*);
- **Taxa de mutação**: fixa em 10% para todas as configurações.

A taxa de mutação fixa elimina interações entre parâmetros que poderiam mascarar o efeito das estratégias de controle populacional, permitindo uma comparação mais justa.

3.2. Estratégias de Controle Populacional

Quatro estratégias foram avaliadas:

- **Substituição total**: todos os indivíduos são substituídos a cada geração.
- **Substituição parcial**: uma fração fixa da população é substituída por novos descendentes.
- **Elitismo**: um subconjunto dos melhores indivíduos é preservado integralmente.
- **Controle por idade (GAVaPS)**: cada indivíduo recebe um *lifetime* proporcional à sua aptidão e é removido quando o ultrapassa, produzindo uma população de tamanho dinâmico.

Nas estratégias de população fixa, utiliza-se uma população de 100 indivíduos. Para o controle por idade, o mesmo valor é utilizado como população inicial, permitindo variação ao longo da execução.

Tabela 1. Configurações avaliadas por estratégia de controle populacional

Estratégia	Parâmetros	Valores
Substituição total	Tamanho da população	100
	Taxa de mutação	10%
Substituição parcial	Fração substituída	25%
	Tamanho da população	100
Elitismo	Fração de elite	10%
	Tamanho da população	100
Controle por idade	Lifetime	$L_{\min} = 1$, $L_{\max} = 5$
	Taxa de reprodução	0,5
	População inicial	100

3.3. Controle por Idade no Contexto Experimental

O controle por idade utilizado segue o modelo descrito na Seção 2. A cada iteração:

1. as idades dos indivíduos são incrementadas;

2. indivíduos cuja idade excede seu *lifetime* são eliminados;
3. seja S o número de sobreviventes, gera-se

$$N_{\text{novos}} = \lfloor \tau \cdot S \rceil ,$$

com $\tau = 0{,}5$, produzindo descendentes por seleção, cruzamento e mutação.

Assim, o tamanho da população torna-se dinâmico:

$$\text{Pop}(t+1) = S + N_{\text{novos}}.$$

Caso $S = 0$, ocorre reinicialização completa da população.

3.4. Desenho Experimental

Para cada estratégia e cada valor de n, realizam-se **100 execuções independentes**, utilizando sementes distintas na inicialização dos geradores de números aleatórios. A repetição independente é fundamental para reduzir variância e permitir inferências confiáveis sobre o desempenho estocástico de cada abordagem.

Critério de parada

Uma mudança central deste trabalho é o uso de um critério de parada exclusivamente baseado no **número total de avaliações da função de aptidão**. Esse critério é amplamente adotado em experimentos científicos com algoritmos evolutivos por refletir diretamente o esforço computacional empregado na busca [Eiben and Smith 2015].

O limite máximo de avaliações é calculado a partir do tamanho da população inicial (denotado por P) e do tamanho do problema n, segundo:

$$\text{limite_avaliacoes} = 2 \cdot P \cdot n.$$

Esse cálculo define um orçamento fixo e igualitário de esforço computacional para todas as estratégias, independentemente de diferenças no custo por geração ou da variação do tamanho populacional no controle por idade. A execução é encerrada somente quando:

- uma solução válida (aptidão zero) é encontrada; ou
- o limite de avaliações é atingido.

O número de gerações executadas passa a ser uma **métrica observada**, e não um critério de parada.

Métricas coletadas

Para cada execução, registram-se:

- **Indicador de sucesso**: se a solução foi encontrada antes do limite;
- **Total de avaliações realizadas**;
- **Número de gerações efetivamente executadas**;

- **Tempo total de execução**;
- **Diversidade populacional média** e **entropia média**, quando aplicável.

Essas métricas permitem avaliar simultaneamente a eficácia (taxa de sucesso), a eficiência (avaliações até a solução) e o comportamento evolutivo (dinâmica populacional e diversidade) das diferentes estratégias de controle populacional.

4. Desenvolvimento

Esta seção descreve a implementação prática do estudo, desde a organização modular da ferramenta experimental até a condução automatizada das execuções e o processamento dos resultados utilizados na análise apresentada na Seção 5.

4.1. Arquitetura da Ferramenta Experimental

A implementação foi construída como uma *ferramenta de experimentação* em `Python`, estruturada em módulos independentes e de responsabilidade bem definida. Essa modularização facilita a reprodutibilidade, a substituição de componentes e a adição de novas estratégias ou cenários experimentais.

A arquitetura geral organiza-se nos seguintes blocos:

- **Núcleo do algoritmo genético**: implementa o laço evolutivo, mantém o estado da população, controla o contador de avaliações e aplica os operadores genéticos em cada iteração.
- **Módulo de representação e aptidão**: implementa a codificação em permutação e a função de aptidão para o problema das *n*-rainhas, incluindo uma versão altamente otimizada com `Numba` para acelerar a avaliação de conflitos diagonais.
- **Módulo de operadores genéticos**: reúne a lógica de seleção, cruzamento (*OX*) e mutação (*swap*), todos implementados de forma vetorizada para minimizar custos de memória e maximizar paralelismo.
- **Módulo de controle populacional**: implementa as quatro estratégias comparadas — substituição total, substituição parcial, elitismo e controle por idade — de forma isolada, garantindo que o restante do algoritmo permaneça invariável entre cenários.
- **Módulo de experimentos**: responsável por orquestrar execuções repetidas, aplicar o critério de parada baseado no número de avaliações, registrar métricas e armazenar os resultados em formato `.csv` e `.json`.
- **Módulo de registro e persistência**: implementa rotinas de logging e serialização, garantindo rastreabilidade completa das execuções (semente utilizada, parâmetros, duração, avaliações realizadas).

Essa separação possibilita que o mesmo núcleo do AG seja testado com diferentes políticas de substituição, alterando-se apenas o módulo de controle populacional. Como consequência, qualquer diferença observada nos resultados é atribuível à estratégia de controle — e não ao restante do algoritmo.

4.2. Implementação dos Operadores Genéticos

Os operadores genéticos seguem rigorosamente as definições apresentadas na Seção 3. Cada indivíduo é um vetor permutacional de tamanho n, no qual o valor na coluna i indica a linha ocupada pela rainha.

A função de aptidão avalia exclusivamente conflitos diagonais. Sua implementação inclui:

- versões sequencial e paralela otimizadas com `Numba`;
- buffers pré-alocados para evitar realocação de memória;
- vetorização completa das operações de contagem.

A seleção é realizada por torneio binário vetorizado. O cruzamento *OX* é aplicado em lote, reduzindo o overhead de chamadas repetidas. A mutação por troca também é vetorizada e utiliza um kernel compilado pelo `Numba` para troca in-place de múltiplos indivíduos em paralelo.

Essas otimizações garantem que o esforço computacional entre as estratégias seja comparável, minimizando vieses relacionados ao custo interno dos operadores.

4.3. Implementação das Estratégias de Controle Populacional

As quatro estratégias avaliadas compartilham operadores e função de aptidão, diferindo apenas na forma como atualizam a população:

- **Substituição total**: toda a população é substituída a cada geração.
- **Substituição parcial**: uma fração fixa da população é substituída; os indivíduos preservados são selecionados *aleatoriamente*, conforme regra estabelecida neste trabalho.
- **Elitismo**: preserva-se uma fração fixa, composta exclusivamente pelos melhores indivíduos.
- **Controle por idade (GAVaPS)**: cada indivíduo recebe um *lifetime* calculado via função bilinear dependente da aptidão; indivíduos cuja idade ultrapassa o limite são removidos; o tamanho da população varia dinamicamente a cada iteração.

No controle por idade, a população inicial é de 100 indivíduos, porém o tamanho pode encolher ou crescer a cada iteração, seguindo:

$$\text{Pop}(t+1) = S + \lfloor 0{,}5\, S \rceil\,.$$

Esse comportamento é implementado sem comprometer a comparabilidade entre estratégias, pois o critério de parada depende apenas do número de avaliações da aptidão, não do número de gerações.

4.4. Execução dos Experimentos

A execução de cada cenário segue cinco etapas principais:

1. **Inicialização**: a população é gerada aleatoriamente e os contadores internos (idade, lifetime e avaliações) são zerados.
2. **Laço evolutivo**: seleção, cruzamento, mutação e avaliação dos novos indivíduos são aplicados repetidamente.
3. **Aplicação da política de controle populacional**: a população é atualizada de acordo com a estratégia em estudo.
4. **Critério de parada**: a execução é encerrada quando:

   $$\text{quant_avaliacoes} \geq 20 \cdot P \cdot n,$$

   onde P é o tamanho da população inicial (100 neste estudo). Se uma solução ótima (aptidão zero) surgir antes desse limite, a execução também é encerrada.

5. **Registro das métricas**: ao final, são salvos em arquivo o número total de avaliações, o tempo total, a quantidade de gerações executadas, a semente, e um indicador de sucesso.

A escolha de utilizar apenas avaliações como critério de parada garante orçamento computacional idêntico entre as estratégias, mesmo quando o número de indivíduos avaliados por geração é variável, como ocorre no controle por idade.

4.5. Organização dos Dados e Pré-processamento

Cada execução gera um registro individual no formato `.csv`, contendo:

- tamanho do problema n;
- estratégia de controle populacional;
- semente usada;
- avaliações totais realizadas;
- número de gerações executadas;
- tempo total;
- indicador de sucesso.

Antes da análise estatística, realiza-se uma etapa de pré-processamento que inclui:

- **validação de registros**, removendo execuções incompletas;
- **cálculo de métricas derivadas**, como taxa de sucesso, avaliações médias até a solução e desempenho relativo;
- **agrupamento por estratégia e tamanho** n, com geração automática de tabelas e gráficos.

Esse fluxo garante independência total entre a implementação do AG e a análise dos dados, permitindo a replicação do estudo apenas pela adição de novos arquivos de saída e reexecução dos scripts de pré-processamento.

5. Resultados e Discussão

Esta seção apresenta e discute os resultados obtidos nos experimentos descritos na Seção 3. Todas as estratégias foram avaliadas sob as mesmas condições: mesmo AG base, mesmos operadores e, principalmente, o mesmo critério de parada, definido exclusivamente pelo limite total de avaliações da função de aptidão:

$$\text{limite_avaliacoes} = \lfloor 20 \cdot N \cdot n \rfloor,$$

onde N é o tamanho da população e n é o tamanho do tabuleiro. Dessa forma, cada estratégia dispõe do mesmo orçamento computacional, eliminando vieses causados por limites de gerações ou comportamentos internos específicos.

Os resultados exibem valores médios obtidos a partir de múltiplas execuções independentes, o que permite analisar padrões gerais de convergência, qualidade e eficiência. As análises foram organizadas nos seguintes eixos:

1. avaliações médias consumidas;
2. geração da melhor solução;
3. gerações médias;
4. melhor aptidão média;
5. taxa de sucesso;
6. tempo médio de execução.

5.1. Avaliações médias por n

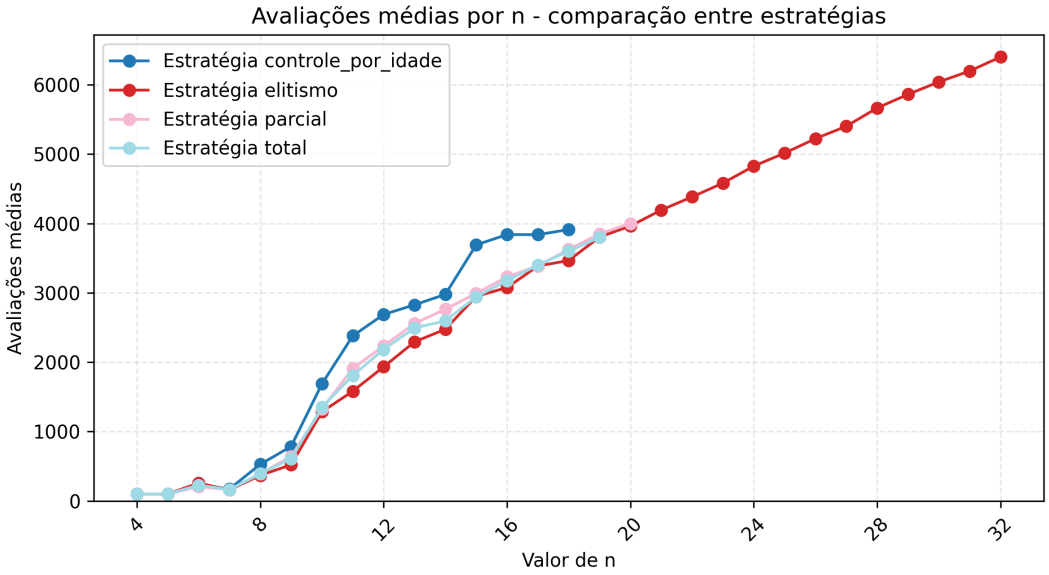


Figura 2. Avaliações médias consumidas por n.

As avaliações médias refletem o quanto cada estratégia utilizou do orçamento total antes de encerrar a execução. Os resultados mostram um comportamento consistente:

- Para instâncias pequenas ($n \leq 12$), todas as estratégias consomem quantidades semelhantes de avaliações, o que indica que o problema ainda é simples.
- Entre $n = 12$ e $n = 18$, as estratégias **controle por idade**, **substituição parcial** e **substituição total** tendem a encerrar antes do limite, indicando maior eficiência evolutiva nessas faixas.
- O **elitismo** apresenta crescimento quase linear, utilizando consistentemente mais avaliações e se aproximando do limite em instâncias maiores. Isso indica convergência mais lenta, embora mais estável.

Esse comportamento sugere que estratégias que preservam diversidade (controle por idade e substituições aleatórias) conseguem explorar mais rapidamente regiões promissoras do espaço de busca, enquanto o elitismo tende a explorar mais profundamente, porém de forma mais lenta.

5.2. Geração da melhor solução

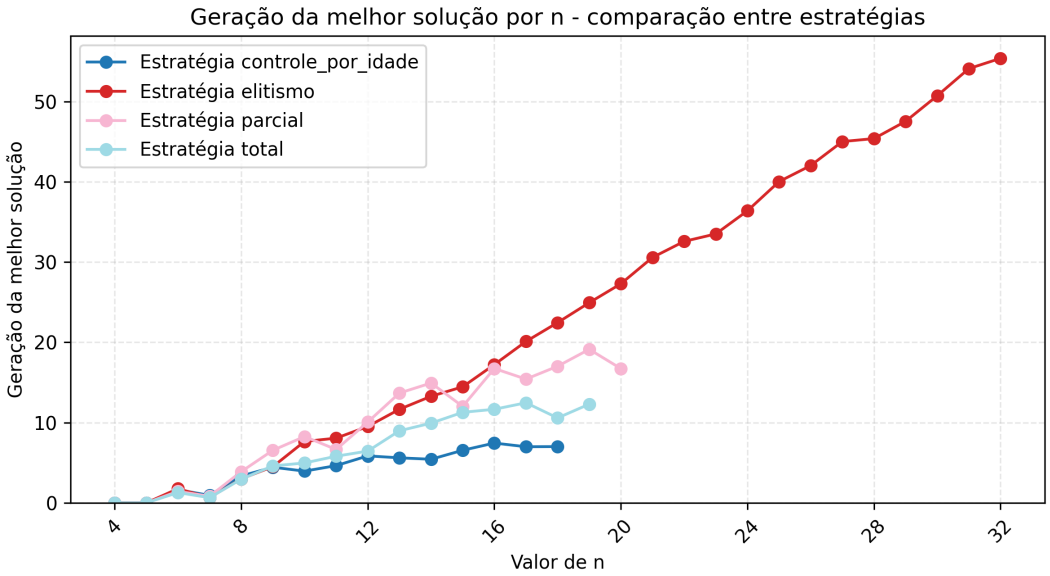


Figura 3. Geração média da melhor solução.

A geração em que a melhor solução é encontrada indica **velocidade inicial de convergência**. Os padrões observados são claros:

- O **controle por idade** encontra boas soluções muito cedo e de forma estável até $n \approx 18$, graças ao mecanismo de lifetime, que retém indivíduos promissores sem impedir a renovação constante.
- O **elitismo** encontra suas melhores soluções de forma progressivamente mais tardia conforme n aumenta. A forte pressão seletiva acelera a convergência final, mas reduz a exploração inicial.
- As estratégias de **substituição parcial** e **total** apresentam maior variabilidade, incluindo oscilações que indicam instabilidade inicial e perda precoce de diversidade.

5.3. Gerações médias

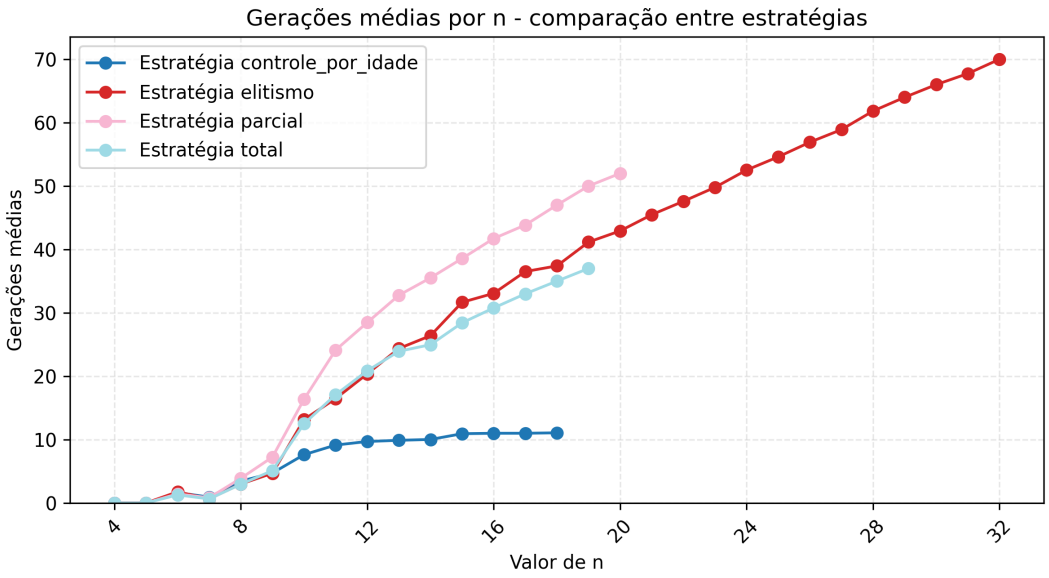


Figura 4. Gerações médias executadas por n.

O número médio de gerações executadas reflete a **profundidade evolutiva** alcançada antes do término.

- O **controle por idade** mantém consistentemente o menor número de gerações até $n \approx 20$, indicando ciclos evolutivos mais produtivos.
- O **elitismo** apresenta um aumento pronunciado, chegando a mais de 70 gerações para $n = 32$. A população tende a se estabilizar parcialmente nas primeiras gerações, exigindo muito mais etapas para refinar soluções.
- As estratégias **total** e **parcial** crescem rapidamente em número de gerações, mas sem ganho de qualidade correspondente — evidenciando travamento precoce em ótimos locais.

5.4. Melhor aptidão média

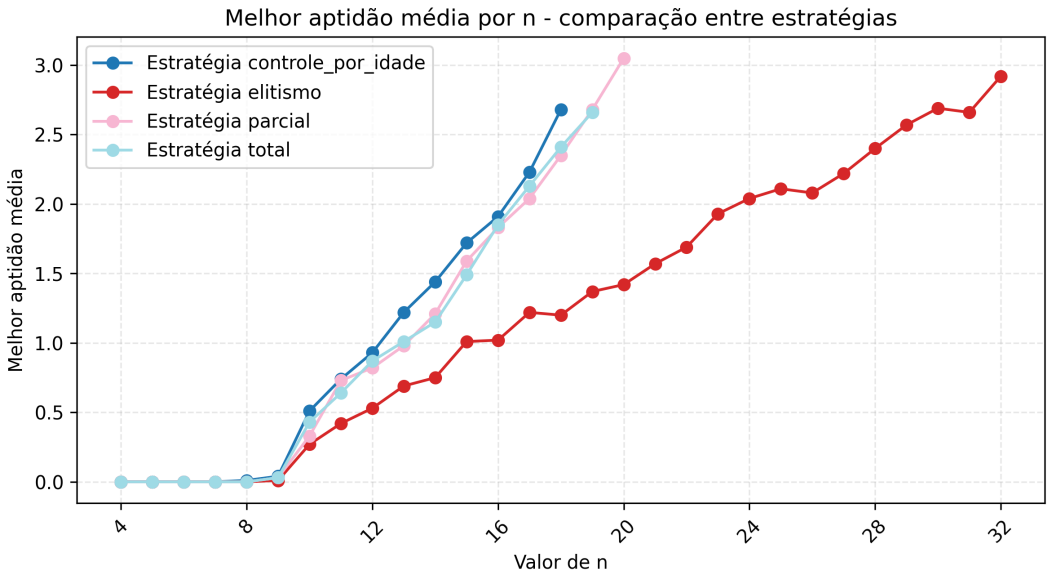


Figura 5. Melhor aptidão média encontrada por n.

A melhor aptidão média representa a **qualidade evolutiva global** das soluções encontradas.

- Para $n \leq 12$, todas as estratégias encontram soluções ótimas com grande consistência.
- A partir de $n > 15$, as estratégias **total** e **parcial** deixam de alcançar valores ótimos e degradam progressivamente, resultado da rápida perda de diversidade.
- Entre $15 \leq n \leq 22$, o **controle por idade** obtém as melhores soluções médias entre todas as estratégias.
- Para $n > 25$, o **elitismo** passa a liderar, encontrando soluções mais próximas do ótimo — embora à custa de mais tempo e menor taxa de sucesso.

5.5. Taxa de sucesso

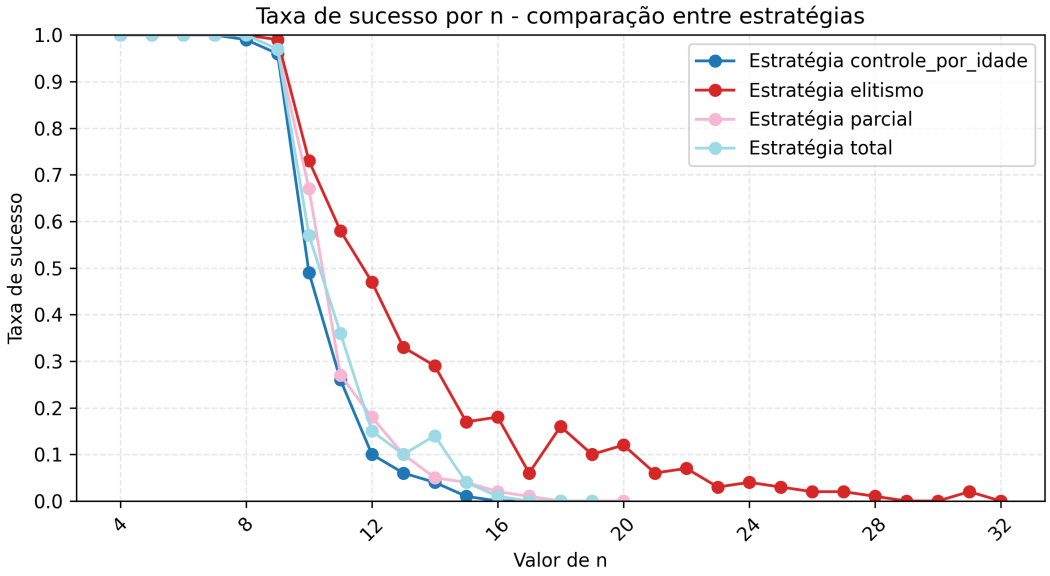


Figura 6. Taxa de sucesso por n.

A taxa de sucesso corresponde à proporção de execuções que encontraram uma solução válida antes do limite de avaliações.

- Todas as estratégias apresentam **100% de sucesso até** $n = 10$.
- As estratégias **total** e **parcial** colapsam rapidamente a partir de $n \geq 14$.
- O **controle por idade** mantém sucesso elevado até $n = 18$, diminuindo gradualmente.
- O **elitismo** apresenta a melhor taxa de sucesso geral, ainda encontrando soluções em instâncias próximas de $n = 32$.

5.6. Tempo médio de execução

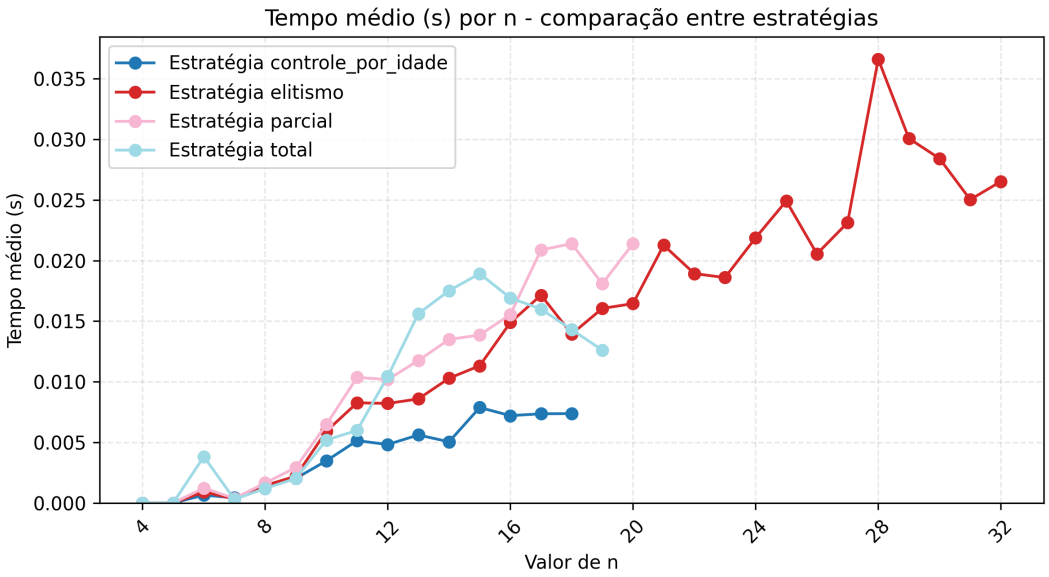


Figura 7. Tempo médio por execução.

O tempo médio de execução depende tanto da complexidade do problema quanto da profundidade evolutiva atingida.

- As estratégias **parcial**, **total** e **controle por idade** apresentam tempos baixos e relativamente estáveis.
- O **elitismo** é significativamente mais lento em instâncias maiores, ultrapassando 0,03s para os maiores valores de n testados.
- O aumento no tempo é diretamente proporcional ao número de gerações executadas — como observado nos gráficos anteriores.

5.7. Síntese geral

Os resultados globais permitem uma caracterização clara das estratégias:

- **Elitismo**: Melhor taxa de sucesso e melhores soluções em instâncias grandes, mas com maior custo temporal e convergência mais lenta.
- **Controle por idade (GAVaPS)**: Melhor equilíbrio geral entre tempo, qualidade e sucesso. Excelente para instâncias médias.
- **Substituição parcial e total**: Adequadas apenas para instâncias pequenas; perdem diversidade rapidamente e falham em instâncias moderadas e grandes.

De maneira geral, os resultados confirmam que o **controle populacional** não é um detalhe operacional, mas sim um componente determinante para o comportamento global de um AG, influenciando diretamente a convergência, o custo computacional e a capacidade de escapar de ótimos locais.

6. Conclusão e Trabalhos Futuros

6.1. Conclusões Gerais

Este trabalho investigou o impacto de diferentes estratégias de controle populacional em algoritmos genéticos aplicados ao problema das *n*-rainhas, utilizando um critério de parada unificado baseado exclusivamente no número máximo de avaliações da função de aptidão. A implementação modular desenvolvida permitiu a execução sistemática e reprodutível dos experimentos, possibilitando comparar de maneira justa as quatro abordagens estudadas: substituição geracional, substituição parcial, elitismo e controle por idade (modelo inspirado no GAVaPS de Michalewicz).

Os resultados demonstraram que o mecanismo de controle populacional influencia profundamente o comportamento evolutivo do algoritmo, afetando diretamente a taxa de sucesso, a qualidade das soluções, a estabilidade da busca e a eficiência do processo evolutivo. Entre as estratégias avaliadas, o **elitismo** destacou-se em instâncias maiores devido à sua maior capacidade de preservar bons indivíduos ao longo das avaliações, mesmo exigindo mais etapas evolutivas para alcançar soluções de qualidade. Por outro lado, o **controle por idade** apresentou o melhor equilíbrio global, combinando boa capacidade exploratória, manutenção de diversidade e rapidez na convergência em instâncias de complexidade moderada.

As abordagens de **substituição geracional** e **substituição parcial** mostraram-se adequadas apenas para instâncias menores, perdendo eficiência rapidamente à medida que o valor de n aumenta. Essa perda de desempenho está fortemente relacionada à baixa capacidade de preservar diversidade e ao rápido colapso estrutural da população.

De modo geral, os experimentos evidenciam que políticas de substituição não devem ser tratadas como detalhes implementacionais, mas sim como componentes centrais da dinâmica do algoritmo genético. Os resultados reforçam também que o controle por idade — com seu mecanismo de renovação gradual e preservação dinâmica de indivíduos — constitui uma alternativa flexível e competitiva em problemas combinatórios desafiadores.

6.2. Limitações do Estudo

Embora os resultados obtidos sejam consistentes, algumas limitações devem ser consideradas:

- **Escopo limitado do domínio**: o estudo foi conduzido exclusivamente no problema das *n*-rainhas. Apesar de ser um benchmark clássico, suas características específicas podem favorecer ou prejudicar determinadas estratégias.
- **Parâmetros fixados**: taxas de mutação, tamanho populacional, pressão seletiva e demais parâmetros foram mantidos constantes ao longo dos experimentos. Outras configurações podem alterar significativamente o comportamento das estratégias.
- **Ausência de paralelização**: todo o experimento foi realizado em ambiente sequencial. Estratégias com populações maiores poderiam se beneficiar substancialmente de execuções paralelas.
- **Análise estatística restrita**: embora médias e tendências tenham sido avaliadas, análises estatísticas formais — como testes de hipótese ou intervalos de confiança — não foram exploradas profundamente.

Essas limitações não invalidam os resultados, mas abrem espaço para investigações complementares.

6.3. Trabalhos Futuros

Com base nas conclusões e limitações identificadas, diversos caminhos promissores podem ser explorados em trabalhos futuros:

- **Aplicar as estratégias a outros problemas**: como Caixeiro Viajante, Sudoku, timetabling ou otimização contínua, verificando a generalidade dos padrões observados.
- **Aprofundar variantes do GAVaPS**: explorar mecanismos avançados de ajuste de idade, regras adaptativas de reprodução ou variação dinâmica do tamanho populacional, conforme proposto por Michalewicz.
- **Realizar análises estatísticas formais**: utilizando ANOVA, testes não paramétricos, bootstrap e intervalos de confiança para validar rigorosamente as diferenças entre estratégias.
- **Aprimorar desempenho computacional**: integrando paralelização em GPU, Numba, multiprocessing ou frameworks como Ray, permitindo experimentar populações maiores e limites maiores de avaliação.
- **Explorar estratégias híbridas**: como combinar controle por idade com elitismo, mutação adaptativa ou mecanismos de preservação explícita de diversidade.
- **Desenvolver um painel visual**: criando uma interface interativa para acompanhar a evolução das populações, curvas de aptidão e métricas avançadas em tempo real.

Essas possibilidades podem ampliar significativamente a compreensão do comportamento evolutivo dos algoritmos genéticos, além de aumentar a aplicabilidade prática das estratégias estudadas.

Referências

Ali, H. et al. (2023). Performance comparison of genetic algorithms with traditional search techniques on the n-queen problem. In *Proceedings of ICIT*.

Bell, J. and Stevens, B. (2009). A survey of known results and research areas for n-queens. *Discrete Mathematics*, 309(1):1–31.

De Jong, K. A. (1975). *An Analysis of the Behavior of a Class of Genetic Adaptive Systems*. PhD thesis, University of Michigan. Ph.D. thesis, referência fundacional dos GAs.

de Lacerda, E. G. M. and de Leon Ferreira de Carvalho, A. C. P. (1999). Introdução aos algoritmos genéticos. In *Sistemas inteligentes: aplicações a recursos hídricos e sistemas ambientais*. Ed. Universidade / UFRGS / ABRH, Porto Alegre.

Eiben, A. E. and Smith, J. E. (2015). *Introduction to Evolutionary Computing*. Natural Computing Series. Springer, Berlin Heidelberg, 2 edition.

Farhan, A. S., Tareq, W. Z., and Awad, F. H. (2015). Solving n queen problem using genetic algorithm. *International Journal of Computer Applications*, 122(12).

Goldberg, D. E. (1989). *Genetic Algorithms in Search, Optimization and Machine Learning*. Addison-Wesley.

Harris, C. R., Millman, K. J., van der Walt, S. J., Gommers, R., Virtanen, P., Cournapeau, D., Wieser, E., Taylor, J., Berg, S., Smith, N. J., et al. (2020). Array programming with NumPy. *Nature*, 585(7825):357–362.

Holland, J. H. (1975). *Adaptation in Natural and Artificial Systems*. University of Michigan Press.

Hunter, J. D. (2007). Matplotlib: A 2d graphics environment. *Computing in Science & Engineering*, 9(3):90–95.

Michalewicz, Z. (1994). Genetic algorithms with varying population size. In *Proceedings of the First IEEE Conference on Evolutionary Computation*, volume 1, pages 115–119. IEEE.

Mitchell, M. (1996). *An Introduction to Genetic Algorithms*. MIT Press.

Python Software Foundation (2025a). *The Python Language Reference, version 3.x*. Python Software Foundation. Acesso em: 28 jun. 2025.

Python Software Foundation (2025b). *Python Module Reference: random — generate pseudo-random numbers*. Python Software Foundation, versão 3.13.5 edition. Acesso em: 28 jun. 2025.

Richter, H. and Berlezi, R. (2012). Algoritmo genético aplicado ao problema das n rainhas. *Anais da UNIJUÍ*. Relatório técnico disponível em proceedings.sbmac.

Sarkar, U. and Nag, S. (2017). An adaptive genetic algorithm for solving n-queens problem. *arXiv preprint*.

Spears, W. M. (1993). Crossover or mutation? In Whitley, D., editor, *Foundations of Genetic Algorithms 2*, pages 221–237. Morgan Kaufmann.

Whitley, D. (1994). A genetic algorithm tutorial. *Statistics and Computing*, 4(2):65–85.